Lab 6

Asynchronous Receiver

- Waits for the line to go from high (idle) to low. The low signal signifies the start bit.
- Using the agreed upon baud rate (also oversamples to not miss anything) it captures the next 8 bits as the data.
- It checks if the bit after the 8 data bits is a high signal. If it’s not, it goes back to idle state and does not pulse the data ready signal.
- It uses a state machine to implement this interaction. There is a state for idle, a state for each data bit, and a state for the stop bit.

Asynchronous Transmitter

- Uses the same scheme as the receiver but it does not have to oversample.
- It takes the 8 data bits and shifts them until the 8 bits are transmitted over the line.
- It sets the line low for the start bit and sets it back high for the end bit. There are states for all bits. There is an extra state for the stop bit, presumably to give the receiver extra time.

char2seg

- The input for this module are 8 bits representing ASCII keys.
- Based on the ASCII 8 bit values generated by the keyboard, an 8 bit output is chosen for the HEX display that best represents the key pressed.
- Some keys generate the same output on the HEX display because the display was not made to be able to represent them.

hello_FSM

- The FSM used to detect if the sequence of ASCII bits received over the UART line represent the word ‘hello’ is implemented using a Mealy model.
- There are 5 states, pressing the correct order will bring the FSM back to it’s initial state but also outputs a 1.
- Going from the last state to the initial state is the only way to output a high signal.
- The state transitions ensure that pressing the letter ‘h’ does not in fact bring us back to the initial state, but rather back to the second state since the initial state is for if the user has not yet input anything or has input the wrong first letter.
- A negative edge (to avoid race conditions) reset signal is assigned to KEY0. Pressing the button at anytime will immediately change the state to the initial state. For example, after typing in ‘hell’, pressing the reset button, and then pressing ‘o’ will not result in a high output.

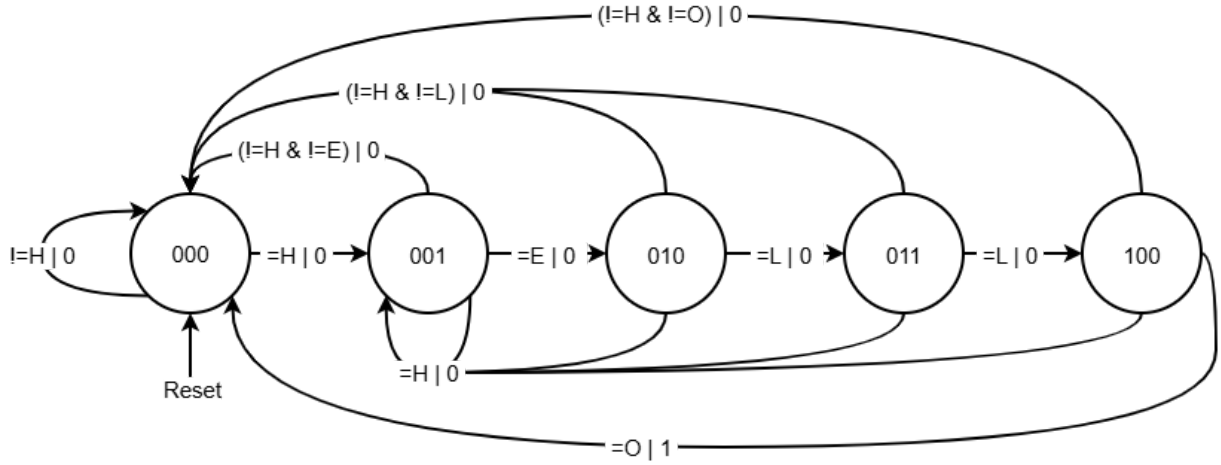


Fig 1 - FSM for ‘hello’ detector.

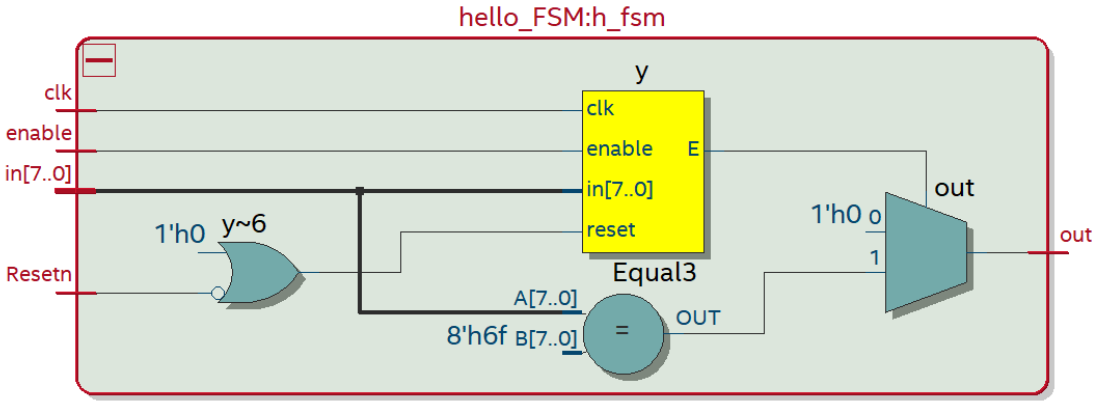


Fig 2 - Quartus RTL view for hello_FSM.

hello_blinker

- The high output generated by the hello_FSM is received by the hello_blinker module.
- The module has a state machine within a state machine.
- The first state is blinking or not blinking. The blinking state is reached once the high output from the hello_FSM is received.
- An accumulator, using the baud rate generator provided, accumulates to a specific value resulting in approximately 3 seconds time interval before transitioning back to the not blinking state.
- The value for the accumulator depends on the baudrate provided to the accumulator clk signal and the target wait interval.
- The second FSM is idle until the blinking state for the first FSM is entered.
- Using a separate accumulator similar to the 3 second accumulator, the second FSM switches between putting ‘hello’ on the HEX array and putting nothing. It changes states every half second.

Entire System

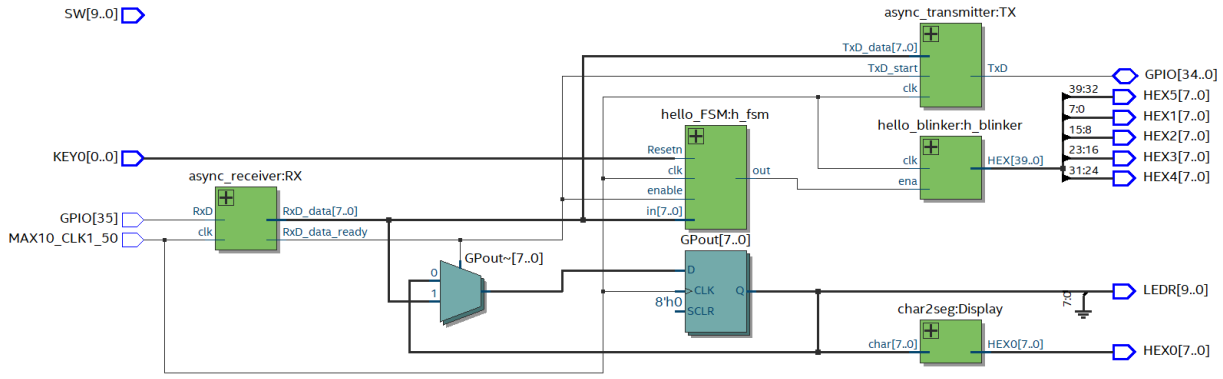


Fig 3 - Quartus RTL view of entire system

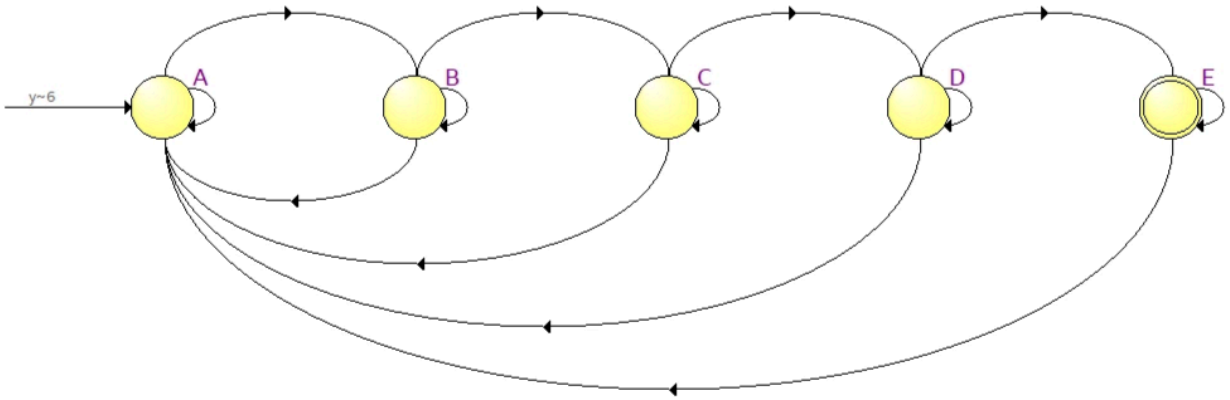


Fig 4 - Entire system state machine generated by Quartus. I believe this is a Moore state machine also I can't be sure how Quartus does works in this regard. My guess is the the double circle on state E signifies a high output. The state transitions that point back to the same state are likely a result of the transient minor states the entire system experiences in between these major states.